FH Salzburg	**Fachhochschule Salzburg**
	Fachrichtung: **Informationstechnik & System-Management**

Datenbanksysteme - Laborprotokoll

Titel	**Library**

Jahrgang	**Authoren**
ITS-2026	Lastowicka Susanne Lux Alexander

Bemerkungen

Für eine Vollständige Auflistung der Befehle und Diagramme, folgen Sie den Links.

Inhalt

1 ER-MODELL 3

1.1 Kommentare 4

2 DDL 4

3 INSERT STATEMENTS 6

3.1 Kommentare 6

4 TRIGGER 11

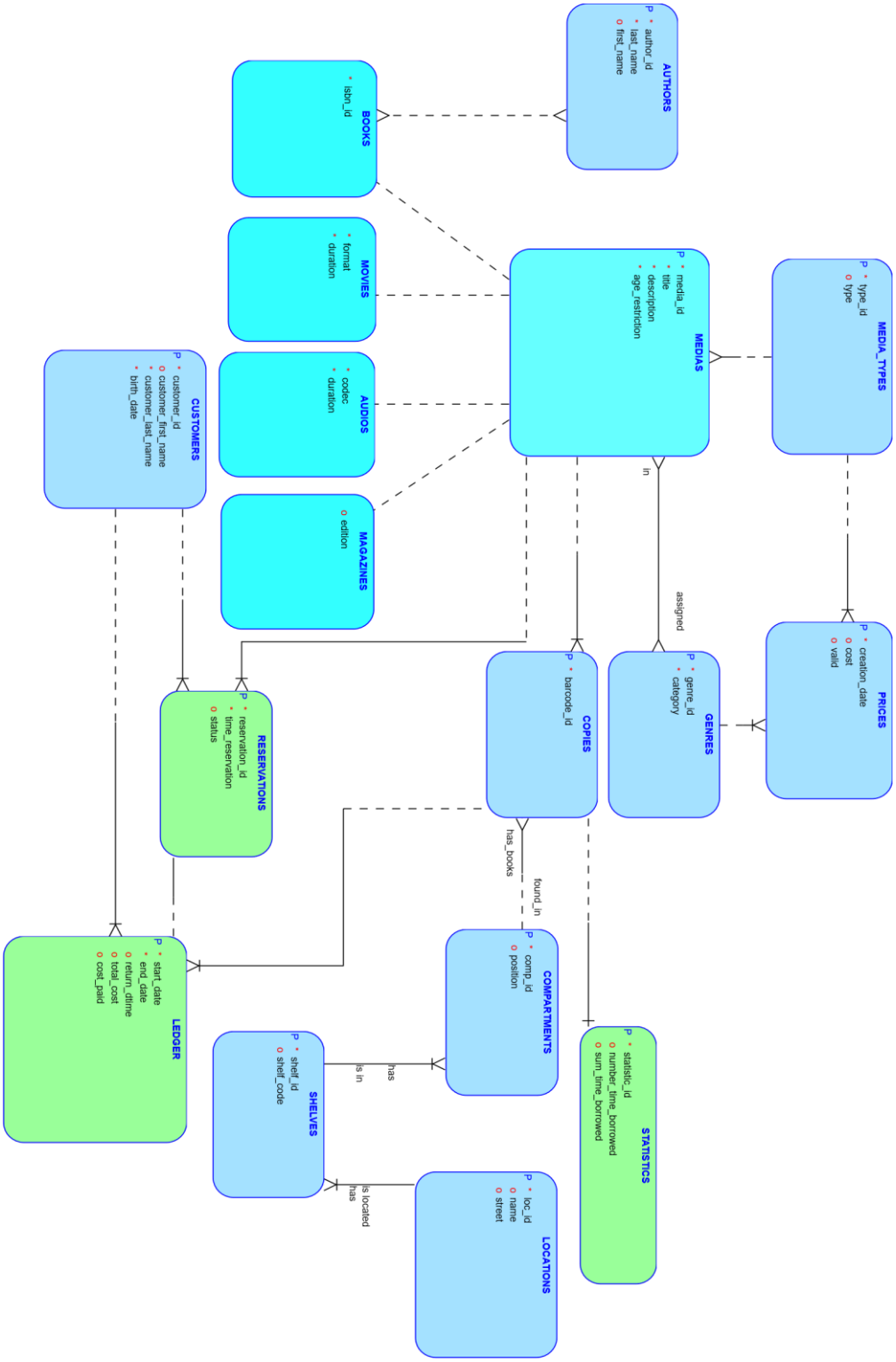
5 CONSTRAINTS 12

6 USE CASES 12

7 JOINS UND ABFRAGEN (ANWENDUNG) 17

8 DISKUSSION DER ERGEBNISSE 18

1 ER-Modell


AUTHORS
P * author_id
* last_name
O first_name
BOOKS
* isbn_id
MEDIA_TYPES
P * type_id
O type
MEDIAS
P * media_id
* title
* description
* age_restriction
MOVIES
* format
* duration
AUDIOS
* codec
* duration
MAGAZINES
O edition
CUSTOMERS
P * customer_id
O customer_first_name
* customer_last_name
* birth_date
PRICES
P * creation_date
O cost
O valid
GENRES
P * genre_id
* category
in
assigned
COPIES
P * barcode_id
RESERVATIONS
P * reservation_id
* time_reservation
O status
has_books
found_in
COMPARTMENTS
P * comp_id
O position
STATISTICS
P * statistic_id
O number_time_borrowed
O sum_time_borrowed
LEDGER
P * start_date
* end_date
O return_dtime
O total_cost
O cost_paid
SHELVES
P * shelf_id
O shelf_code
has
is in
LOCATIONS
P * loc_id
O name
O street
is located
has

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Diagramme/ER_Diagram_Lastowicka_Lux_V3.pdf

1.1 Kommentare

Grün: Bewegungsdaten

Blau: Stammdaten

Türkis: Vererbung(-shierachie)

Kann keine Reservierungen und Ausleihen geben ohne einen Kunden und Kopien. Compartments sind von Shelves abhängig, Shelves sind von Location abhängig. Statistiken sind von Kopien abhängig. Kosten sind von Genre und Medientyp abhängig.
Der Preis wird als Kostenmatrix betrachtet und der Primärschlüssel setzt sich zusammen aus type_id, genre_id und dem creation_date. Das creation_date ist deshalb Teil des Schlüssels, falls die Kosten geupdated werden.
Customer hat einen optionalen Vornamen, weil es sein kann, dass eine Familie etwas ausleiht (nur Nachname).
Bei Location reicht der Name der Library plus die Straße aus, weil nur wichtig ist, aus welcher Library man das Buch holen müsste.
Bei der Vererbung wurde eine vertikale Partitionierung, um NULL values zu vermeiden, da die Attribute sehr unterschiedlich sind und Beziehungen über den Ober-typ Media geregelt werden können.

2 DDL

```
CREATE TABLE MEDIA (
    media_id NUMBER PRIMARY KEY,
    title VARCHAR2(100) NOT NULL,
    description VARCHAR2(500),
    age_restriction NUMBER,
    media_type_id NUMBER NOT NULL,
    CONSTRAINT media_type_fk FOREIGN KEY (media_type_id) REFERENCES MEDIA_TYPE(media_type_id)
);

CREATE TABLE BOOKS (
    media_id NUMBER PRIMARY KEY,
    isbn VARCHAR2(50) NOT NULL,
    CONSTRAINT books_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id)
);
```

```
-- 6. TRANSAKTIONEN (LEDGER & RESERVATION)
CREATE TABLE LEDGER (
    ledger_id NUMBER PRIMARY KEY,
    customer_id NUMBER NOT NULL,
    copy_id NUMBER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    return_date DATE,
    total_cost NUMBER(9,2),
    cost_paid NUMBER(9,2),
    genre_id NUMBER,        -- Teil der Kostenverknüpfung
    media_type_id NUMBER,   -- Teil der Kostenverknüpfung
    CONSTRAINT ledger_customer_fk FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id),
    CONSTRAINT ledger_copy_fk FOREIGN KEY (copy_id) REFERENCES COPY(copy_id),
    CONSTRAINT fk_ledger_cost_matrix FOREIGN KEY (genre_id, media_type_id)
        REFERENCES COST_EVALUATION (genre_id, media_type_id)
);
```

V1 DDL:
https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-3_DDL_pretty.sql

```
CREATE TABLE PRICES (
    MEDIA_TYPES_type_id INTEGER NOT NULL,
    GENRES_genre_id     INTEGER NOT NULL,
    creation_date       DATE NOT NULL,
    cost                NUMBER(9,2),
    valid               CHAR(1),
    CONSTRAINT PRICES_PK PRIMARY KEY (MEDIA_TYPES_type_id, GENRES_genre_id, creation_date),
    CONSTRAINT PRICES_GENRES_FK FOREIGN KEY (GENRES_genre_id) REFERENCES GENRES (genre_id),
    CONSTRAINT PRICES_MEDIA_TYPES_FK FOREIGN KEY (MEDIA_TYPES_type_id) REFERENCES MEDIA_TYPES (type_id)
);
```

```
CREATE TABLE LEDGER (
    start_date            DATE NOT NULL,
    end_date              DATE NOT NULL,
    return_dtime          DATE,
    total_cost            NUMBER(9,2),
    cost_paid             NUMBER(9,2),
    CUSTOMERS_customer_id INTEGER NOT NULL,
    COPIES_barcode_id     VARCHAR2(15) NOT NULL,
    COPIES_media_id       INTEGER NOT NULL,
    CONSTRAINT LEDGER_PK PRIMARY KEY (CUSTOMERS_customer_id, COPIES_barcode_id, COPIES_media_id, start_date),
    CONSTRAINT LEDGER_CUSTOMERS_FK FOREIGN KEY (CUSTOMERS_customer_id) REFERENCES CUSTOMERS (customer_id),
    CONSTRAINT LEDGER_COPIES_FK FOREIGN KEY (COPIES_barcode_id, COPIES_media_id)
        REFERENCES COPIES (barcode_id, MEDIAS_media_id)
);
```

Version 2 DDL:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-3_2_DDL_pretty.sql

3 INSERT STATEMENTS

3.1 Kommentare

Zuerst sollten eigenständige Entitäten ohne Abhängigkeiten angelegt werden, wie z. B. Media_type, Autoren, Genre, Location und Kunden.

Anschließend werden die abhängigen Entitäten erstellt, und zwar in der Reihenfolge ihrer Abhängigkeiten. Das bedeutet beispielsweise, dass eine Location bereits existieren muss, bevor Shelves angelegt werden können. Ebenso müssen Media_type und Genre vorhanden sein, bevor eine Cost_evaluation sinnvoll definiert werden kann.

Kunden:

```
INSERT INTO CUSTOMER (first_name, last_name, birth_date) VALUES ('Alexander','Lux',TO_DATE('01.01.1900','DD.MM.YYYY'));
INSERT INTO CUSTOMER (first_name, last_name, birth_date) VALUES ('Susanne','Lastowicka',TO_DATE('02.02.1902','DD.MM.YYYY'));
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	BIRTH_DATE
1	Alexander	Lux	1900-01-01 00:00:00.000
2	Susanne	Lastowicka	1902-02-02 00:00:00.000
3	Dominik	Zach	1903-03-03 00:00:00.000
4	Max	Mustermann	2002-02-02 00:00:00.000

Version 2:

```
INSERT INTO CUSTOMERS (customer_first_name, customer_last_name, birth_date)
VALUES ('Alexander', 'Lux', TO_DATE('01.01.1950', 'DD.MM.YYYY'));
```

Medien Typ:

```
INSERT INTO FHS52423.MEDIA_TYPE
(MEDIA_TYPE_ID, NAME)
VALUES(0, 'BOOK');
```

Medien:

```
--MEDIA
INSERT INTO FHS52423.MEDIA
(TITLE, DESCRIPTION, AGE_RESTRICTION, MEDIA_TYPE_ID)
VALUES('Star Wars Episode I', 'A long time ago in a galaxy far, far away', 6, 1);
```

```
-- CONCRETE MEDIA TYPE (BOOK; MOVIE; AUDIO; MAGAZINE)
--BOOK
INSERT INTO FHS52423.BOOKS (MEDIA_ID, ISBN)
VALUES (
    (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = '1984'),
    '9-9-9-9'
);
```

MEDIA_ID	TITLE	DESCRIPTION	AGE_RESTRICTION	MEDIA_TYPE_ID
1	Star Wars Episode I	A long time ago in a galaxy far, far away	6	1
2	1984	1984, geschrieben von 1946 bis 1948	16	0
3	Hiraeth	homesickness	0	2
4	Vogue	Druckerzeugnis	0	3
7	TEST	test	18	0

Version 2:

Medium erstellen und dem Untertype den Aktuellen Sequenzzähler mitgeben.

```
-- Insert a Book
INSERT INTO MEDIAS (title, description, age_restriction, MEDIA_TYPES_type_id)
VALUES ('1984', 'Written 1948', 16, 1);

INSERT INTO BOOKS (media_id, isbn_id)
VALUES (MEDIA_SEQ.CURRVAL, '978-3-16-148410-0');
```

Author:

```
--AUTHOR

INSERT INTO FHS52423.AUTHORS
( FIRST_NAME, LAST_NAME)
VALUES('Sapphó', 'Ψάπφω');
```

```
--AUTHOREN DIE ETWAS GESCHRIEBEN HABEN UND DEREN MEDIEN
--WRITTEN_BY

INSERT INTO FHS52423.WRITTEN_BY (AUTHOR_ID, MEDIA_ID)
VALUES (
    (SELECT AUTHOR_ID FROM FHS52423.AUTHORS WHERE LAST_NAME = 'Orwell' AND FIRST_NAME = 'George'),
    (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = '1984')
);
```

Media-Genre:

```
-- Star Wars SCI-FI
INSERT INTO FHS52423.MEDIA_GENRE (MEDIA_ID, GENRE_ID)
VALUES (
    (SELECT MEDIA_ID FROM FHS52423.MEDIA WHERE TITLE = 'Star Wars Episode I'),
    (SELECT GENRE_ID FROM FHS52423.GENRE WHERE NAME = 'SCI-FI')
);
```

Version 2:

```
-- Genres
INSERT INTO GENRES (category) VALUES ('SCI-FI');
```

Standort->Regale->Fächer:

```
-- LOCATION
INSERT INTO FHS52423.LOCATION
( NAME, STREET)
VALUES('Stadtbibliothek Bischofshofen', 'Kinostraße 4');
```

```
--shelves
--BISCHOFSHOFEN
INSERT INTO FHS52423.SHELVES
( LOCATION_ID, SHELF_CODE)
VALUES( 1, 'REGAL A1');
```

```
-- COMPARTMENT

INSERT INTO FHS52423.COMPARTMENT
( SHELF_ID, "POSITION")
VALUES( 1, 'Oben');
```

Version 2:

```
--create new entry with nextval
INSERT INTO COMPARTMENTS (comp_id, SHELVES_shelf_id, SHELVES_LOCATIONS_loc_id, position)
VALUES (COMPARTMENT_SEQ.NEXTVAL, 'S1', 2, 'Top Shelf');
```

Kopien:

```
INSERT INTO FHS52423.COPY (barcode, media_id, compartment_id)
VALUES (
    'BC002',
    (SELECT media_id FROM MEDIA WHERE title = 'Star Wars Episode I'),
    2
);
```

COPY_ID	BARCODE	MEDIA_ID	COMPARTMENT_ID	CREATED_AT
1	BC001	3	2	2026-03-23 17:59:53.000
3	BC002	1	2	2026-03-23 18:00:52.000
4	BC003	2	1	2026-03-23 18:00:56.000
5	BC004	1	4	2026-03-23 18:00:58.000

Version 2:

```
-- Copy of Interstellar in the Sci-Fi Section
INSERT INTO COPIES (barcode_id, MEDIAS_media_id, COMPARTMENTS_comp_id, COMPARTMENTS_shelf_id, COMPARTMENTS_loc_id)
VALUES ('BC-INT-001',
        (SELECT media_id FROM MEDIAS WHERE title = 'Interstellar'),
        (SELECT comp_id FROM COMPARTMENTS WHERE position = 'Top Shelf' AND SHELVES_LOCATIONS_loc_id = 2),
        'S1', 2);
```

Preise:

```
--COST_EVAL

-- BOOK
INSERT INTO FHS52423.COST_EVALUATION
(MEDIA_TYPE_ID, COST, VALID, CREATION_DATE, GENRE_ID)
VALUES(0, 1.5, 'Y', SYSDATE, 1);
```

Version 2:(Preise)

MEDIA_TYPES_TYPE_ID	GENRES_GENRE_ID	CREATION_DATE	COST	VALID
1	1	2026-04-13 18:45:12.000 GMT+	1,5	Y
1	5	2026-04-13 18:45:17.000 GMT+	1,2	Y
1	3	2026-04-13 18:45:19.000 GMT+	2	Y
2	1	2026-04-13 18:45:24.000 GMT+	3,5	Y
2	6	2026-04-13 18:45:26.000 GMT+	3	Y
3	2	2026-04-13 18:45:27.000 GMT+	1	Y
4	4	2026-04-13 18:45:30.000 GMT+	0,8	Y

Ausleihen:

```
--LEDGER
INSERT INTO FHS52423.LEDGER
(CUSTOMER_ID, COPY_ID, START_DATE, END_DATE, TOTAL_COST, MEDIA_TYPE_ID, GENRE_ID)
VALUES (
    (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lux'),
    (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
    SYSDATE - 10,
    SYSDATE - 5,
    (SELECT MAX(COST) FROM COST_EVALUATION WHERE media_type_id = 0),
    0,
    (SELECT genre_id FROM COST_EVALUATION
     WHERE media_type_id = 0 AND cost = (SELECT MAX(cost) FROM COST_EVALUATION WHERE media_type_id = 0)
     AND ROWNUM = 1)
);
```

ID	KUNDE	MEDIUM_TITEL	BARCODE	TYP	GENRE	AUSGELIEHEN	RUECKGABE_SOLL	PREIS
2	Dominik Zach	Hiraeth	BC001	AUDIO	Dystopie	14.03.2026	18.03.2026	10 €
1	Alexander Lux	Star Wars Episode I	BC004	MOVIE	SCI-FI	13.03.2026	18.03.2026	1,5 €

Version 2:

	START_DATE	END_DATE	RETURN_DTIME	TOTAL_COST	COST_PAID	CUSTOMERS_CUSTOMER_ID	COPIES_BARCODE_ID	COPIES_MEDIA_ID
1	2026-04-11 17:56:35.000 GM	2026-04-18 17:56:35.000	2026-04-18 17:57:35.000	4,5	[NULL]	3	BC-INT-001	4
2	2026-04-11 00:00:00.000 GM	2026-04-18 00:00:00.000	[NULL]	4,5	[NULL]	3	BC-INT-001	4
3	2026-04-11 00:00:00.000 GM	2026-04-18 00:00:00.000	[NULL]	1,2	[NULL]	1	BC-HOBBIT-01	3

```
INSERT INTO LEDGER (start_date, end_date, CUSTOMERS_customer_id, COPIES_barcode_id, COPIES_media_id, total_cost)
SELECT
    TRUNC(SYSDATE) - 2,
    TRUNC(SYSDATE) + 5,
    (SELECT customer_id
     FROM CUSTOMERS
     WHERE customer_last_name = 'Lux'
     FETCH FIRST 1 ROW ONLY),
    'BC-HOBBIT-01',
    (SELECT m.media_id FROM MEDIAS m WHERE m.title = 'The Hobbit'),
    (
        SELECT MAX(p.cost)
        FROM PRICES p
        JOIN MEDIA_GENRE mg
            ON p.GENRES_genre_id = mg.GENRES_genre_id
        WHERE mg.MEDIAS_media_id = m.media_id
        AND p.MEDIA_TYPES_type_id = m.MEDIA_TYPES_type_id
        AND p.valid = 'Y'
    );
```

Reservierungen:

```
INSERT INTO FHS52423.RESERVATION (
    customer_id,
    copy_id,
    reservation_date,
    status
)
VALUES (
    (SELECT customer_id FROM CUSTOMER WHERE last_name = 'Lastowicka'),
    (SELECT copy_id FROM COPY WHERE barcode = 'BC004'),
    SYSDATE,
    'AKTIV'
);
```

Reservierung besitzen den Fremdschlüssel, um Null werte zu vermeiden, weil Reservierungen optional sind.

RESERVATION_ID	TIME_RESERVATION	CUSTOMER_ID	MEDIA_ID	STATUS	BARCODE_ID	START_DATE
3	2026-04-13 17:58:03.000	1	3	PENDING	BC-INT-001	2026-04-11 00:00:00.000

Statistica:

STATISTIC_ID	NUMBER_TIME_BORROWED	SUM_TIME_BORROWED	COPIES_BARCODE_ID	COPIES_MEDIAS_MEDIA_ID
4	1	2	BC-HOBBIT-01	3
5	3	11	BC-INT-001	4

```sql
INSERT INTO STATISTICS (
    number_time_borrowed,
    sum_time_borrowed,
    COPIES_barcode_id,
    COPIES_medias_media_id
)
SELECT
    COUNT(*) as number_time_borrowed,
    SUM(TRUNC(NVL(return_dtime, SYSDATE)) - TRUNC(start_date)) as sum_time_borrowed,
    COPIES_barcode_id,
    COPIES_media_id
FROM LEDGER
GROUP BY COPIES_barcode_id, COPIES_media_id;
```

Liste der ersten Inserts:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2._INSERTS.sql

Liste aller neuen Inserts:

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2_2._INSERTS.sql

4 TRIGGER

Mit dem Gedanken, dass sich nicht-natürliche Schlüssel automatisch erhöhen bei Eingabe von Objekten, haben wir Auto-Inkrement eingebaut.

```oracle
-- Auto-Increment Triggers
CREATE OR REPLACE TRIGGER media_bi BEFORE INSERT ON MEDIA FOR EACH ROW BEGIN
    IF :NEW.media_id IS NULL THEN SELECT media_seq.NEXTVAL INTO :NEW.media_id FROM dual; END IF; END;
/
CREATE OR REPLACE TRIGGER author_bi BEFORE INSERT ON AUTHORS FOR EACH ROW BEGIN
    IF :NEW.author_id IS NULL THEN SELECT author_seq.NEXTVAL INTO :NEW.author_id FROM dual; END IF; END;
/
```

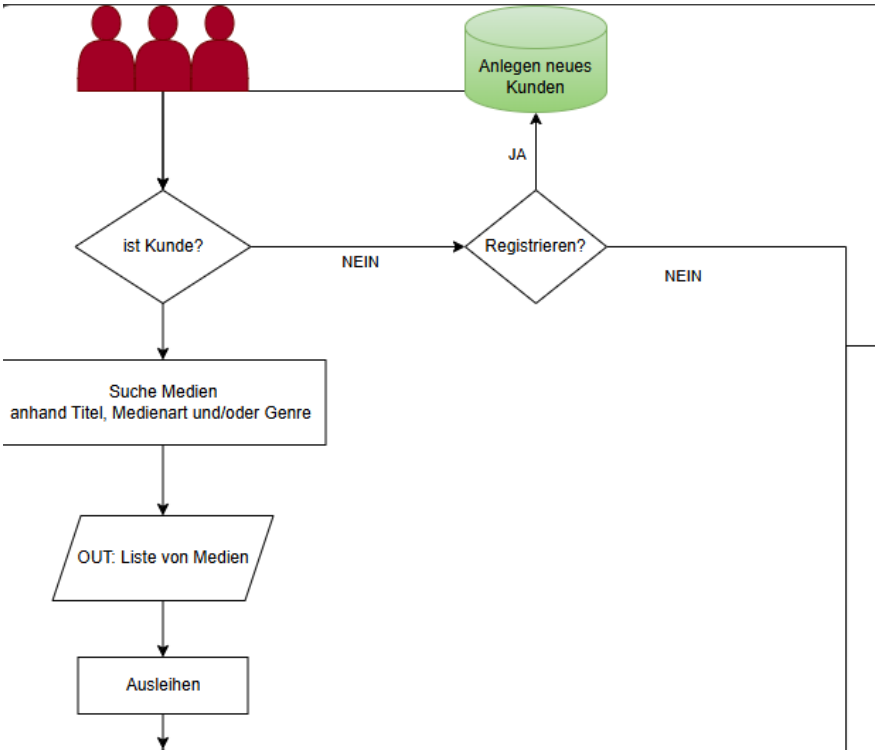
5 CONSTRAINTS

```oracle
CREATE TABLE COPY (
    copy_id NUMBER PRIMARY KEY,
    barcode VARCHAR2(50) UNIQUE NOT NULL,
    media_id NUMBER NOT NULL,
    compartment_id NUMBER NOT NULL,
    created_at DATE DEFAULT SYSDATE,
    CONSTRAINT copy_media_fk FOREIGN KEY (media_id) REFERENCES MEDIA(media_id),
    CONSTRAINT copy_compartment_fk FOREIGN KEY (compartment_id) REFERENCES COMPARTMENT(compartment_id)
);
```

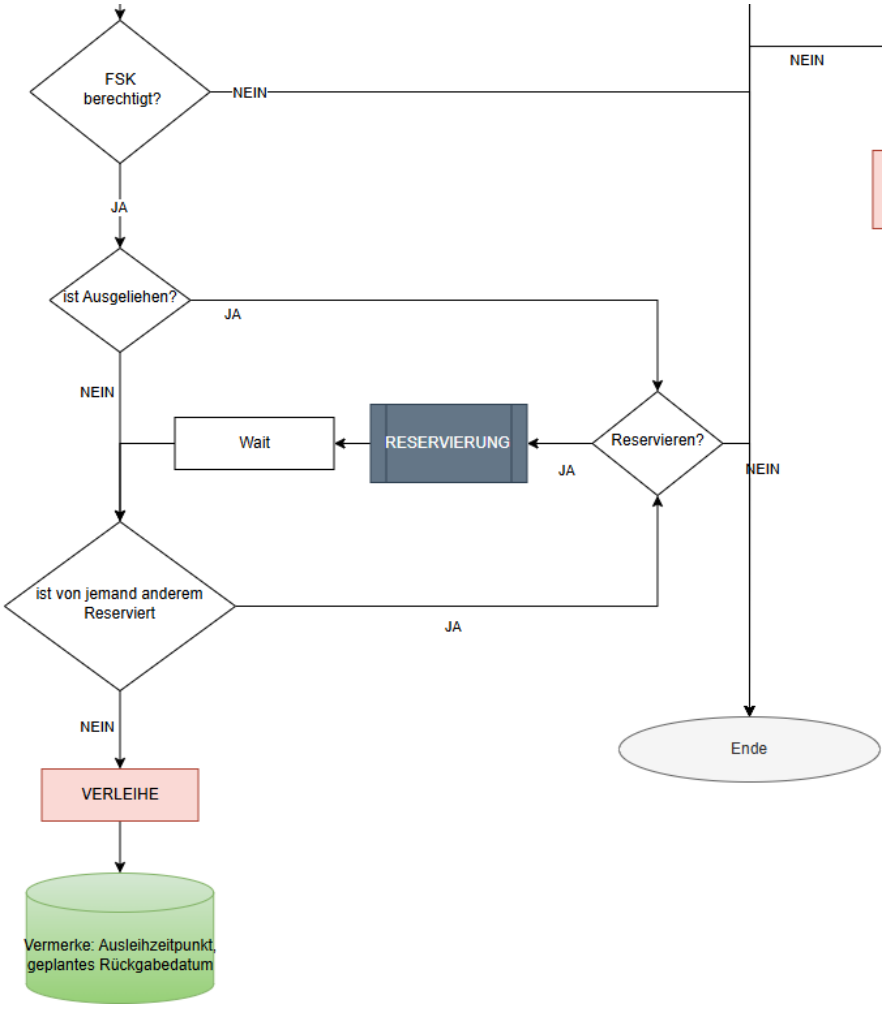
6 Use Cases

Ausleihen

Möchte ein Kunde etwas ausleihen, muss er zuerst ins System eingetragen werden (Mindestens mit Nachnamen und Geburtsdatum). Danach kann er durch Eingabe von entweder Titel, Medienart und/oder Genre nach Medien suchen, diese werden ausgegeben.

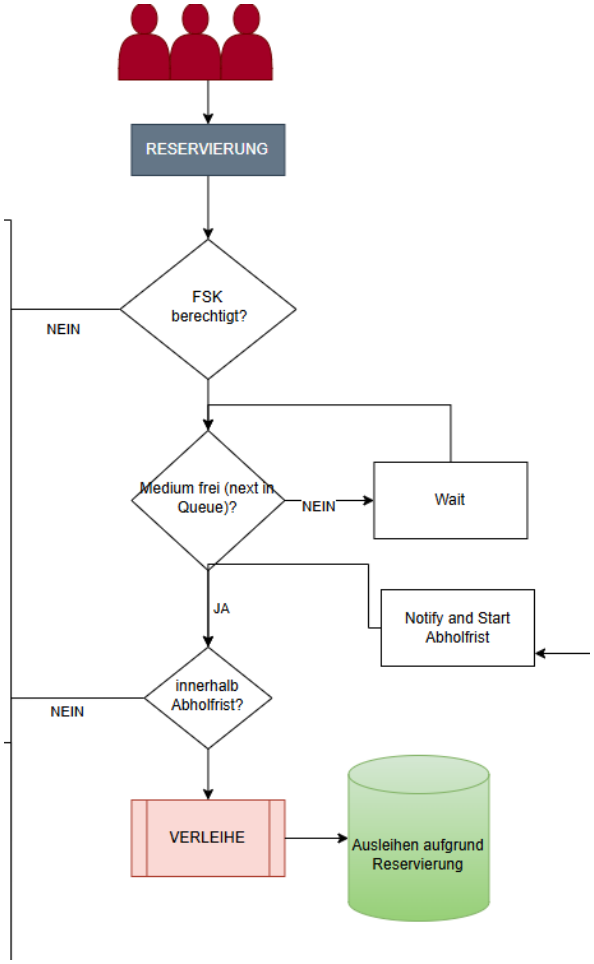


Beim Ausleihen wird zuerst überprüft, ob man alt genug ist. Dann wird geprüft, ob bereits alle Exemplare verliehen sind, wenn ja besteht die Möglichkeit es zu reservieren. Danach wird die Reservierungsliste durchgegangen, bis man an der Reihe ist. Die älteste Reservierung wird ausgewählt. Dem Kunden wird das Exemplar zugewiesen. In der Datenbank wird gespeichert, wer, was, wann und bis wann ausleiht (Kunde, Medium,Zeitraum).



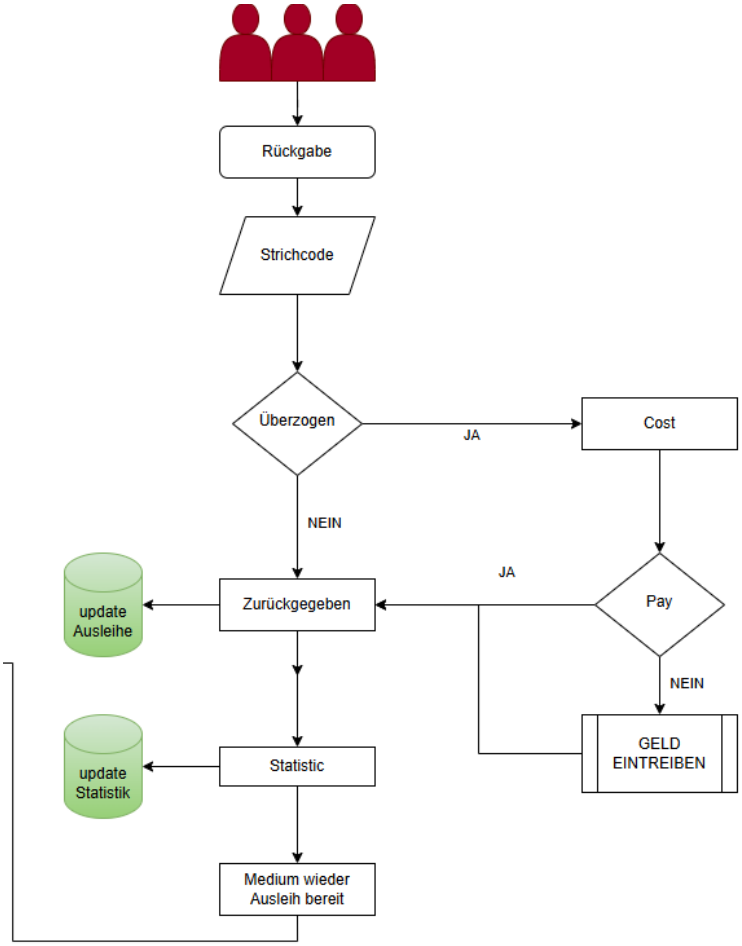
Reservieren

Möchte ein Kunde etwas Reservieren, wird zuerst überprüft, ob er alt genug für das Medium ist. Ist das der Fall wird in der Tabelle vermerkt: wer, was, wann reserviert (Kunde, Medium, Datum), weiteres kommt er in eine Warteschlange (Reihenfolge der Kunden, nach Reservierungsdatum aufsteigend sortiert), bis das nächste Exemplar für den Kunden frei ist. Er kann sich seinen Artikel, innerhalb von 3 Tagen abholen, ansonsten verfällt die Reservierung. Beim Ausleihen wird in der Reservierungstabelle der Verweis auf die Ausleihe gespeichert.


RESERVIERUNG
FSK berechtigt?
NEIN
Medium frei (next in Queue)?
NEIN
Wait
JA
Notify and Start Abholfrist
innerhalb Abholfrist?
NEIN
VERLEIHE
Ausleihen aufgrund Reservierung

Rückgabe

Bei der Rückgabe eines Buches wird aufgrund des Strichcodes überprüft, um welches Exemplar es sich handelt. Dann wird aufgrund dem im Ledger festgelegten start_date überprüft, ob die Rückgabefrist eingehalten wurde oder nicht. Bei Nichteinhaltung wird der Preis für das Überziehen berechnet und der Kunde muss diesen zahlen. Wird nicht gezahlt (was nicht explizit als Fall genannt wird), wird das Geld auf andere Art angefordert. Dann wird das Buch zurückgegeben, die STATISTICS updaten für dieses Exemplar die Zeit, und die Anzahl der Ausleihen wird um 1 erhöht. Der LEDGER wird auch geupdatet (return time, cost, cost paid), dadurch kann die Kopie auch direkt wieder ausgeliehen werden.



https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Diagramme/Use_Cases.drawio.svg

7 JOINS und Abfragen (Anwendung)

Rückgabe (Ausschnitt)

```oracle
-- ==========================================
-- RÜCKGABE
-- ==========================================

-- 1. Medium/Kopie identifizieren (Schritt: "Strichcode")
-- Prüft, ob der Barcode existiert und holt die Stammdaten
SELECT c.*, m.TITLE
FROM COPIES c
JOIN MEDIAS m ON c.Medias_MEDIA_ID = m.MEDIA_ID
WHERE c.BARCODE_ID = :barcode
FOR UPDATE WAIT 5;

-- 2. Kosten berechnen & im Ledger speichern (Schritt: "Überzogen?" & "Cost")
-- Dieser Befehl berechnet die Gebühr und schreibt sie in TOTAL_COST.
-- GREATEST(0, ...) verhindert negative Kosten bei rechtzeitiger Abgabe.
-- 'cost'-Wert aus PRICES, der aktuell 'valid' ist.
-- Dieser Wert wird mit der Anzahl der Tage Multipliziert.
UPDATE LEDGER l
SET l.TOTAL_COST = GREATEST(0,
    ROUND(
        (SYSDATE - l.END_DATE) * (
            SELECT p.cost
            FROM PRICES p
            -- Verknüpfung über MEDIAS, um den passenden Preis für die Kopie zu finden
            JOIN MEDIAS m ON (m.MEDIA_TYPES_TYPE_ID = p.type_id AND m.genre_id = p.genre_id)
            JOIN COPIES c ON (c.Medias_MEDIA_ID = m.MEDIA_ID)
            WHERE c.BARCODE_ID = l.COPIES_BARCODE_ID
            AND p.valid <= SYSDATE -- Preis muss bereits gültig sein
            ORDER BY p.valid DESC    -- Das aktuellste Datum zuerst
            FETCH FIRST 1 ROW ONLY
        ), 2)
    )
WHERE l.COPIES_BARCODE_ID = :barcode
AND l.RETURN_DTIME IS NULL;
```

Kundenverwaltung

```oracle
-- ==========================================
-- KUNDENVERWALTUNG & SUCHE
-- ==========================================

-- Prüfen, ob Kunde existiert (ist Kunde?)
SELECT * FROM CUSTOMERS WHERE CUSTOMER_LAST_NAME = :last_name AND BIRTH_DATE = :birth_d;

-- Neuen Kunden anlegen (Anlegen neues Kunden)
INSERT INTO CUSTOMERS (CUSTOMER_FIRST_NAME, CUSTOMER_LAST_NAME, BIRTH_DATE)
VALUES (:first_n, :last_n, :birth_d);

-- Suche Medien (Titel, Genre oder Typ)
SELECT m.*, g.CATEGORY, mt.TYPE
FROM MEDIAS m
JOIN MEDIA_TYPES mt ON m.MEDIA_TYPES_TYPE_ID = mt.TYPE_ID
JOIN GENRES g ON m.MEDIA_ID = g.GENRE_ID
WHERE m.TITLE LIKE %:search% OR g.CATEGORY = :genre OR mt.TYPE = :type;
```

Verleihen

```sql
-- ========================================
-- VERFÜGBARKEIT & VERLEIHE
-- ========================================

-- FSK Prüfung (FSK berechtigt?)
-- Rechnet das Alter in Jahren aus und vergleicht mit age_restriction
SELECT * FROM MEDIAS m
WHERE m.MEDIA_ID = :media_id
AND m.age_restriction <= FLOOR(MONTHS_BETWEEN(SYSDATE, (SELECT birth_date FROM CUSTOMERS WHERE customer_id = :c_id)) / 12);

-- Prüfen, ob eine Kopie aktuell frei ist (ist Ausgeliehen?)
-- Ein Medium ist frei, wenn es eine Kopie gibt, die nicht in einem offenen Ledger-Eintrag steht
SELECT * FROM COPIES c
WHERE c.Medias_MEDIA_ID = :media_id
AND c.BARCODE_ID NOT IN (SELECT COPIES_BARCODE_ID FROM LEDGER WHERE RETURN_DTIME IS NULL)
FOR UPDATE WAIT 5;

-- Verleihen: Neuen Eintrag im Ledger anlegen
INSERT INTO LEDGER (START_DATE, END_DATE, TOTAL_COST, COST_PAID, CUSTOMERS_CUSTOMER_ID, COPIES_BARCODE_ID)
VALUES (SYSDATE, SYSDATE + 21, (SELECT cost FROM PRICES WHERE creation_date = (SELECT MAX(creation_date) FROM PRICES)), 0, :c_id, :barcode);
```

Reservierung

```sql
-- ==========================================
-- RESERVIERUNG
-- ==========================================

-- Neue Reservierung anlegen (Warteschlange)
INSERT INTO RESERVATIONS (TIME_RESERVATION, STATUS, CUSTOMER_ID, MEDIA_ID)
VALUES (SYSDATE, 'PENDING', :c_id, :media_id);

-- Prüfen: Wer ist der nächste in der Schlange? (next in Queue?)
SELECT * FROM RESERVATIONS
WHERE MEDIA_ID = :media_id AND STATUS = 'PENDING'
ORDER BY TIME_RESERVATION ASC
FOR UPDATE SKIP LOCKED -- Überspringt Datensätze, die bereits von anderen in Bearbeitung sind
FETCH FIRST 1 ROW ONLY;

-- Reservierung abschließen (Wenn abgeholt wird)
UPDATE RESERVATIONS
SET STATUS = 'DONE'
WHERE RESERVATION_ID = :res_id;
```

Ist der Kunde, der am längsten in der Reservierungswarteliste ist, an der Reihe, weil ein Exemplar frei geworden ist, und holt dieses nicht innerhalb der 3 Tage ab, verfällt die Reservierung. Das wird rein über die Software geregelt.

Mehrbenutzerbetrieb

Für den Mehrbenutzerbetrieb wird “Read Committed” verwendet. Kritische Blöcke, bei denen es zu Anomalien kommen könnte, werden durch “FOR UPDATE” oder später in der Software behandelt.

https://github.com/LuxAlexander/Laboruebungen/blob/main/Datenbanksysteme/Scripte/Script-2_Anwendung.sql

8 Diskussion der Ergebnisse